Laboratorio 8 - 31.05.2022

Esercizio 1

Si consideri il seguente problema

$$\begin{cases} -\Delta u = f & \text{in } \Omega \\ u = g & \text{on } \partial\Omega \end{cases}. \tag{1}$$

1. A partire dal codice assegnato, all'interno della `function C_main2D.m` riportata di seguito, identificare le righe di codice che eseguono l'assegnamento delle condizioni di Dirichlet.

```
function [errors,solutions,femregion,Matrices,Dati] = C_main2D(TestName,
    nRef)
%% [errors,solutions,femregion,Matrices,Dati] = C_main2D(TestName,nRef)
%==========================================================================
% Solution of the Poisson's problem with linear finite elements
% (non homogeneous Dirichlet boundary conditions)
%==========================================================================
%
%    INPUT:
%          Dati        : (struct)  see C_dati.m
%          nRef        : (int)     refinement level
%
%    OUTPUT:
%          errors      : (struct) contains the computed errors
%          solutions   : (sparse) nodal values of the computed and exact
%                        solution
%          femregion   : (struct) infos about finite elements
%                        discretization
%          Matrices    : (struct) fe stiffness and mass matrices
%          Dati        : (struc)  see C_dati.m
%

warning off;
addpath MeshGeneration
addpath FESpace
addpath BoundaryConditions
addpath Assembly
addpath Postprocessing
```

```
addpath Errors

%==========================================================================
% LOAD DATA FOR TEST CASE
%==========================================================================

Dati = C_dati(TestName);
Dati.nRefinement = nRef;

%==========================================================================
% MESH GENERATION
%==========================================================================

[Region] = C_create_mesh(Dati);

%==========================================================================
% ...
%==========================================================================

[femregion] = C_create_femregion(Dati,Region);

%==========================================================================
% ...
%==========================================================================

[A,b]=C_matrix2D(Dati,femregion);

%==========================================================================
% ...
%==========================================================================

[A,b,u_g] = C_bound_cond2D(A,b,femregion,Dati);

%==========================================================================
% SAVE DATA in THE STRUCT Matrices
%==========================================================================

[Matrices] = C_build_out_matrices(A,b,u_g);

%==========================================================================
% ...
%==========================================================================

uh = A\b;

%==========================================================================
% ...
%==========================================================================

uh = uh + u_g;

%==========================================================================
```

```
% ...
%==========================================================================

[solutions] = C_postprocessing(Dati,femregion,uh);

%==========================================================================
% ERROR ANALYSIS
%==========================================================================

[errors] = C_compute_errors(Dati,femregion,solutions);
```

2. A partire dal codice assegnato scrivere le istruzioni necessarie per assemblare la matrice di rigidezza globale A (sfruttare la `function C_lap_loc.m` fornita nel codice). In particolare si modifichi la function Matlab (nella cartella `Assembly`)

$$[\mathtt{A}, \mathtt{f}] = \mathtt{matrix2D}(\mathtt{Dati}, \mathtt{femregion})$$

   e rinominarla in modo opportuno nella `function C_main2D.m`;

3. Utilizzare il codice implementato per risolvere il problema su una successione di triangolazioni strutturate del dominio $\Omega = (-1, 2) \times (0, 1)$ (si fissi `refinement_vector=[3,4,5,6]` in `C_Dati.m`). Sapendo che $u_{ex} = 2x\cos(\pi x)\sin(2\pi y)$, assegnare in modo opportuno i dati nel file `C_dati.m` e poi calcolare gli errori $L^2(\Omega)$ e $H^1(\Omega)$ (sfruttare la `function C_convergence_test.m`). Riportare i risultati in una tabella e consegnare la figura ottenuta per l'andamento degli errori. Si commenti quanto ottenuto alla luce dei risultati teorici conosciuti.

Importante: Riportare, sul modulo Forms, i comandi Matlab utilizzati per rispondere alla domanda (3).

Esercizio 2

Si consideri il seguente problema

$$u_t - u_{xx} = f(x,t) \qquad x \in (0,1), t \in (0,T] \tag{2}$$
$$u(0,t) = u(1,t) = 0 \qquad t \in [0,T] \tag{3}$$
$$u(x,0) = \sin(\pi x) \qquad x \in (0,1) \tag{4}$$

dove $f(x,t) = (\pi^2 - 1)\, e^{-t} \sin(\pi x)$ e la soluzione esatta è $u(x,t) = e^{-t}\sin(\pi x)$.

1. Si approssimi (2)-(4) utilizzando il metodo delle differenze finite per la discretizzazione spaziale ed il metodo di Crank-Nicolson per la discretizzazione temporale. In particolare si completino le istruzioni nella function Matlab assegnata

$$[\mathtt{x}, \mathtt{t}, \mathtt{u}] = \mathtt{calore}(\mathtt{N}, \mathtt{T}, \mathtt{K}, \mathtt{fun}, \mathtt{u0})$$

dove `N` numero di sottointervalli (di eguale ampiezza) in $(0, 1)$, `T` estremo finale intervallo temporale $(0, T)$, `K` numero di sottointervalli (di eguale ampiezza) in $(0, T)$, `fun` funzione che descrive il termine noto f, `u0` funzione che descrive la condizione iniziale a $t = 0$, `x` vettore dei nodi spaziali, `t` vettore dei nodi temporali, `u` soluzione numerica.

2. Si prenda $T = 1$, $N = 10$ e $K = 10$. Si eseguano $M = 4$ dimezzamenti sia in h che in Δt. Si riportino in una **tabella** i valori ottenuti dell'errore

$$e = \max_k \max_n |u(x_n, t^k) - u_n^k|.$$

Alla luce dei teoremi di convergenza conosciuti si commentino i risultati numerici ottenuti.

Importante: Riportare, sul modulo Forms, i comandi Matlab utilizzati per rispondere alla domanda (2).